

บทเรียนที่ 1

รหัสวิชา 21910-2011

บทบาทของโปรแกรมเชิงวัตถุ



บทเรียนที่ 1

บทบาทของโปรแกรมเชิงวัตถุเบื้องต้น

หัวข้อเรื่อง (Topics)

- 1.1 ความหมายของโปรแกรมเชิงวัตถุ
- 1.2 บทบาทของโปรแกรมเชิงวัตถุ
- 1.3 ความเป็นมาและแนวคิดการเขียน โปรแกรมเชิงวัตถุ
- 1.4 คุณสมบัติของโปรแกรมเชิงวัตถุ
- 1.5 ประโยชน์ของโปรแกรมเชิงวัตถุ

แนวคิดสำคัญ (Main Idea)

การเขียนโปรแกรมเชิงวัตถุหรือ (OOP : Object-Oriented Programming) คือ รูปแบบการเขียนโปรแกรมโดยมอง ทุกอย่าง เป็นวัตถุ ซึ่งสามารถนำมาประกอบกันและนำมาทำงานร่วมกันได้ โดยใช้การแลกเปลี่ยนข้อมูลเพื่อนำมาประมวลผล และ ส่ง ข้อมูลที่ได้ไปให้วัตถุอื่นๆ ที่เกี่ยวข้องเพื่อให้ทำงานต่อไป แม้รูปแบบการเขียนจะค่อนข้างยากและมีความซับซ้อน แต่จะส่งผล ดี ต่อการแก้ไขโปรแกรมในระยะยาว ซึ่งแนวคิดนี้จะแยกปัญหา หรือแยกระบบงานออกเป็น ส่วนย่อยเช่นกัน เพื่อลดความซับซ้อน ให้น้อยลง ส่วนย่อยหรือโปรแกรมย่อยที่ว่า เรียกว่า "Class" ซึ่งภายใน Class จะสร้างตัวแทนขึ้นมาทำงานแทนตนเอง เรียกว่า "Object" ซึ่งเป็นตัวที่เก็บข้อมูลต่าง ๆ รวมกันเป็นวัตถุขึ้นเดียวกัน เพื่อให้ง่ายต่อการเรียกใช้งาน ง่ายต่อการแก้ไข และง่ายต่อการ นำกลับมาใช้ใหม่ของระบบในอนาคต

สมรรถะย่อย (Element of Competency)

แสดงความรู้เกี่ยวกับบทบาทของโปรแกรมเชิงวัตถุเบื้องต้น

จุดประสงค์เชิงพฤติกรรม (Behavioral Objectives)

1. บอกความหมาย คุณสมบัติ และประโยชน์ของโปรแกรมเชิงวัตถุได้ถูกต้อง
2. นำเสนอข้อมูลเกี่ยวกับบทบาทของโปรแกรมเชิงวัตถุได้
3. แสดงเจตคติและกิจนิสัยที่ดีในการปฏิบัติงานด้วยความละเอียด และรอบคอบ
4. บอกแนวทางการนำความรู้เกี่ยวกับบทบาทของโปรแกรมเชิงวัตถุเบื้องต้น ไปประยุกต์ใช้ในงานอาชีพได้อย่างเหมาะสม

ผลลัพธ์การเรียนรู้ (Learning Outcomes)

ประยุกต์ใช้ความรู้เกี่ยวกับบทบาทของโปรแกรมเชิงวัตถุเบื้องต้นตามมาตรฐานด้วยความละเอียด และรอบคอบ

เนื้อหาสาระ (Content)

การเขียนโปรแกรมเชิงวัตถุ เป็นการนำเอาแนวคิดเชิงวัตถุมารวบรวมกลุ่มของเมธอด และแอตทริบิวต์ที่มีลักษณะการทำงานที่คล้ายกันไว้ในที่เดียวกัน เพื่อให้ง่ายต่อการเรียกใช้ ซึ่งในแนวคิดนี้ในชุดคำสั่งของการเขียนโปรแกรมเชิงวัตถุ แทนที่เราจะเขียนคำสั่งเรียงต่อกันไปเรื่อยๆ แต่จะเป็นการนิยามวัตถุหรือสร้างวัตถุขึ้นมา จากนั้นโปรแกรมจะทำงานได้ด้วยตัวเอง ถ้าวัตถุนั้นถูกนิยามขึ้นมาอย่างเหมาะสม แต่ในการเขียนโปรแกรมเชิงวัตถุนั้น จะต้องอาศัยระยะเวลาในการศึกษานานพอสมควร

ดังนั้น ในการเตรียมความพร้อมในการเขียนโปรแกรมเชิงวัตถุ ซึ่งจะต้องทำความเข้าใจในการเขียนโปรแกรมโครงสร้างมาบ้าง โดยเฉพาะอย่างยิ่งนักเขียนโปรแกรมต้องมีความชำนาญในการสร้างวัตถุสมมติที่ทำงานจริงในปัจจุบัน การทำความเข้าใจองค์ประกอบของวัตถุมีความจำเป็นอย่างยิ่งที่ต้องทำความเข้าใจใน ขั้นตอนพื้นฐานการเขียนโปรแกรมเชิงวัตถุ ในการเขียนโปรแกรมเพื่อวิเคราะห์ปัญหานั้น ต้องเข้าใจถึงการที่วัตถุจะสามารถสื่อสารกัน โดยการส่งข้อความระหว่างวัตถุ โดยวัตถุนั้นจะสามารถโต้ตอบกันได้โดยไม่ต้องทราบรายละเอียดของข้อมูลหรือชุดคำสั่งที่เขียนของวัตถุนั้น

1.1 ความหมายของโปรแกรมเชิงวัตถุ

การทำโปรแกรมเชิงวัตถุ หมายถึง วิธีการเขียนโปรแกรมของนักเขียนโปรแกรมรุ่นใหม่ ที่จัดแบ่งการเขียนคำสั่งกันออกเป็นชุดๆ แต่ละชุดเรียกว่า "วัตถุ" (object) แล้วจึงนำเอาชุดคำสั่งแต่ละชุดนั้นมารวมกันเป็นโปรแกรมชุดใหญ่อีกหนึ่งทีหนึ่ง ในบางครั้งยังอาจนำ "วัตถุ" ของโปรแกรมหนึ่งไปรวมกับ "วัตถุ" ของอีกโปรแกรมหนึ่ง แล้วเรียกออกมาใช้ได้เลย ทั้งนี้ทำให้ผู้เขียนโปรแกรมใหม่ไม่จำเป็นต้องเริ่มต้นใหม่ทั้งหมด วิธีการดังกล่าวนี้ช่วยประหยัดเวลาได้มาก พูดให้ง่ายก็คือ ทุกโปรแกรมไม่ต้องเริ่มต้นจากศูนย์ นั่นคือ คำว่า "วัตถุ" นั้นหมายรวมไปถึงภาพหรือกราฟิกด้วย ภาพหนึ่งภาพ เช่น การสร้างวงกลมนั้น เกิดจากการเขียนโปรแกรมโดยใช้สูตรคำนวณเส้นโค้ง ซึ่งจะประกอบด้วยคำสั่งหลายร้อยคำสั่ง แล้วเก็บไว้เป็น "วัตถุ" หนึ่ง ฉะนั้น เมื่อใดที่เราสั่งวาดวงกลม ก็เท่ากับไปเรียก "วัตถุ" นี้มาใช้ หลังจากนั้น หากต้องการต่อเติมเป็นภาพอื่นต่อไปคอมพิวเตอร์ก็จะไปดึงอีก "วัตถุ" หนึ่งมาทำต่อให้

1.2 บทบาทของโปรแกรมเชิงวัตถุ

โปรแกรมเชิงวัตถุ OOP (Object Oriented Programming) เป็นวิธีการเขียนโปรแกรม โดยอาศัยแนวคิดของวัตถุชิ้นหนึ่ง มีความสามารถในการปกป้องข้อมูล และการสืบทอดคุณสมบัติ ซึ่งทำให้แนวโน้มของ OOP ได้รับการยอมรับและได้พัฒนาเมโซระบบท่าย 1 มามายเบนระบบมบททาวนเทรต เบนตาน การเขียนโปรแกรมเชิงวัตถุ OOP : Object Oriented Programming คือหนึ่งในรูปแบบการเขียนโปรแกรมคอมพิวเตอร์ ที่ให้ความสำคัญกับวัตถุ ซึ่งสามารถนำมาประกอบกันและนำมาทำงานรวมกันได้โดยการแลกเปลี่ยนข่าวสารเพื่อนำมาประมวลผลและส่งข่าวสารที่ได้ไปให้ วัตถุอื่น ๆ ที่เกี่ยวข้อง เพื่อให้ทำงาน ต่อไปได้

แนวคิดดั้งเดิมของการเขียนโปรแกรม ก็คือ การแก้ปัญหาโดยใช้คอมพิวเตอร์เป็นเครื่องมือ แนวคิดนี้คล้ายกับการใช้เครื่องคิดเลขในการแก้ปัญหาคณิตศาสตร์ ดังนั้น การพัฒนาตัวแปลภาษาไม่ว่าจะเป็นภาษาเครื่องภาษาแอสเซมบลี ภาษาซี และภาษาอื่น ๆ ก็มีแนวการเขียนโปรแกรมแบบเดียวกัน เพียงแต่เปลี่ยนรูปแบบและกฎเกณฑ์ของภาษาเท่านั้น ด้วยเหตุนี้ จึงได้มีการเสนอแนวคิดใหม่ในการเขียนโปรแกรมที่เรียกว่า การเขียนโปรแกรมเชิงวัตถุ แนวคิดแบบใหม่ที่ใช้ในการการเขียนโปรแกรม ก็คือการเน้นถึงปัญหาและองค์ประกอบของปัญหาเพื่อแก้ปัญหา การเน้นที่ปัญหาและองค์ประกอบของการแก้ปัญหา (Problem Space) จะคล้ายกับการแก้ไข ปัญหาและชีวิตความเป็นอยู่ของมนุษย์ที่จะต้องมี คน สัตว์ สิ่งของ เพื่อแก้ปัญหา มีหน้าที่แก้ปัญหา มากกว่าจะมองที่วิธีการแก้ปัญหานั้น ๆ หรือขั้นตอนในการแก้ปัญหา (Solution Space) ซึ่งเป็นวิธีการเขียนโปรแกรมแบบเก่านั้นเอง อาลัน เคย์ (Alan Kay) เป็นผู้บุกเบิกแนวความคิดในการเขียนโปรแกรมเชิงวัตถุ และเป็นผู้ที่มีส่วนในการพัฒนาตัวแปลภาษา Small Talk ซึ่งเป็นต้นแบบของการเขียนโปรแกรมเชิงวัตถุได้เสนอกฎ 5 ข้อ ที่เป็นแนวทางของภาษาคอมพิวเตอร์เชิงวัตถุ หรือที่เรียกว่า Object Oriented Programming (OOP) ไว้ดังนี้ 1. ทุก ๆ สิ่งเป็นวัตถุ (Everything is an Object) 2. โปรแกรม ก็คือ กลุ่มของวัตถุที่ส่งข่าวสารบอกกันและกันให้ทำงาน (A Program is a Bunch of Object Telling Each Other What to do by Sending Messages)

3. ในวัตถุแต่ละวัตถุจะต้องมีหน่วยความจำและประกอบไปด้วยวัตถุอื่นๆ (Each Object has it's Own Memory Made Up of Other Objects)

4. วัตถุทุกชนิดจะต้องจัดอยู่ในประเภทใดประเภทหนึ่ง (Every Object has a Type)

5. วัตถุที่จัดอยู่ในประเภทเดียวกันย่อมได้รับข่าวสารเหมือนกัน (All objects of a Particular type Can Receive the Same Messages)

แนวคิดแบบ OOP ถ้าเราไม่มองในแง่มุมมองของการเขียนโปรแกรมเพียงอย่างเดียว ให้มองไปในภาพรวม มองไปในสิ่งรอบ ๆ ตัว เราสามารถบอกได้ว่าแนวคิดของ OOP ก็คือ "ธรรมชาติของวัตถุ" หมายความว่า OOP จะมองสิ่งแต่ละสิ่งถือเป็น "วัตถุชิ้นหนึ่ง" (Object) ซึ่งจะมีสีแสดหรือสีเขียวจะขาวหรือส้ม ซึ่งก็คือวัตถุชิ้นหนึ่งเหมือนกันนั่นเองและเราสามารถกำหนดประเภทหรือคลาสให้กับวัตถุเหล่านั้นได้ เช่น วัตถุสีเขียวก็จะมารวมอยู่ในกลุ่มเดียวกันหรือวัตถุที่มีขนาดยาวก็มารวมอยู่ในกลุ่มเดียวกัน เป็นต้น เมื่อ OOP มองทุกสิ่งถือเป็นวัตถุชิ้นหนึ่งแล้ว ก็สามารถกล่าวได้อีกว่า "วัตถุแต่ละอย่างนั้นต่างก็มีลักษณะและวิธีการใช้งานเป็นของตัวเอง" ซึ่งหมายถึงวัตถุแต่ละชนิดหรือแต่ละชิ้นต่างก็มีรูปร่าง ลักษณะ และการใช้งานหรือการกระทำที่แตกต่างกันออกไป ซึ่งเราสามารถเรียกคุณลักษณะของวัตถุนี้ว่า แอตทริบิวต์ (Attribute) และจะเรียกวิธีการใช้งานวัตถุว่า เมธอด (Method) ตัวอย่างเช่น

"ดินสอเป็นวัตถุ ซึ่งมีลักษณะยาวเรียว ภายในมีไส้ถ่านใช้สำหรับเขียน การใช้ดินสอสามารถทำได้ โดยใช้มือจับและเขียนลงบนวัตถุที่รองรับ"

จากประโยคดังกล่าว ซึ่งสามารถจับใจความได้ว่า คุณลักษณะของวัตถุ (Attribute) ก็คือ "ยาวเรียว ภายในเป็นไส้ถ่าน"

ส่วนการใช้งาน (Method) ก็คือ "ใช้มือจับและเขียนลงบนวัตถุที่รองรับ"

จากตัวอย่างดังกล่าว สรุปได้ว่า ถ้าเกิดวัตถุใดมีลักษณะยาวเรียว มีไส้ เป็นถ่าน เมื่อจะใช้งานจะต้องใช้ มือจับและเขียนลงบนวัตถุที่รองรับ ซึ่งสามารถบอกได้เลยว่าสิ่งนั้นก็คือ "ดินสอ" นั่นเองจะเห็นได้ว่าแนวคิดของ OOP นั้นจะต้องมีลักษณะที่คล้ายกับธรรมชาติของสิ่งหนึ่งโดยสามารถแยกสิ่งต่าง ๆ ออกเป็นแต่ละประเภทได้ ถ้านำเอาแนวคิดของ OOP มาใช้ในการเขียนโปรแกรมและในการจัดการข้อมูลนั้นจะเห็นได้ว่าโปรแกรมหรือฟังก์ชันแต่ละตัว ถึงแม้ว่าจะมาจากที่เดียวกันแต่สามารถทำงานในคนละหน้าที่เก็บข้อมูลคนละค่าได้ โดยจะไม่ยุ่งเกี่ยวกับแต่อย่างใด เนื่องจากหลักการเขียนโปรแกรมเชิงวัตถุเป็นแนวคิดแบบใหม่ ดังนั้น การทำงานหลาย ๆ ส่วนของ การเขียนโปรแกรมแบบนี้ ซึ่งยังไม่เป็นที่คุ้นเคยมากนัก จึงควรมีความรู้เบื้องต้นเกี่ยวกับ OOP ดังนี้

การเชื่อมต่อ (Interface)

อินเทอร์เฟซ (Interface) หมายถึง การเชื่อมต่อ ถ้าการเชื่อมต่อระหว่างผู้ใช้กับคอมพิวเตอร์ จะเรียกการเชื่อมต่อนั้นว่า ยูสเซอร์อินเทอร์เฟซ (User Interface) โดยปกติจะหมายถึง ส่วนของหน้าจอที่ผู้ใช้ส่งหรือ ติดต่อให้คอมพิวเตอร์ทำงาน แต่ในการเขียนโปรแกรมเชิงวัตถุ การเชื่อมต่อยังรวมไปถึงวัตถุ (Object) เพราะในวัตถุจะต้องมีอินเทอร์เฟซ ก็คือ การเปลี่ยนแปลงที่เกิดขึ้นภายในวัตถุจะไม่กระทบต่ออินเทอร์เฟซ ดังนั้นภายในวัตถุผู้เขียนคำสั่งสามารถดัดแปลง แก้ไข หรือเพิ่มเติมได้ตลอดเวลา นอกจากนี้ ภายในวัตถุยังสามารถเก็บค่าต่าง ๆ โดยไม่จำเป็นต้องทราบกลไกการทำงานภายใน เมื่อใช้ก็เพียงแค่เรียกส่วนของอินเทอร์เฟซ (หรือ ที่เรียกว่า Method) โดยการส่งข่าวสาร (Message) ไปยังวัตถุที่ต้องการ ตัวอย่างเช่น การทำงานของโทรศัพท์กับรีโมท ซึ่งการใช้งานของโทรศัพท์มีดังต่อไปนี้ คือ เปิด ปิด เปลี่ยนช่องเมื่อผู้ใช้ต้องการเปิดสัญญาณ โดยกด ปุ่มสัญลักษณ์เปิดส่งไปยังตัวรับโทรศัพท์ หลังจากนั้น โทรศัพท์ก็จะทำงานในกลไกของการเปิด ซึ่งเราไม่ต้อง รับรู้ถึงวงจรที่อยู่ภายในเลย โดย เปิด ปิด และเปลี่ยนช่อง ก็คือ Method ของโทรศัพท์ เป็นต้น

การซ่อนรายละเอียด (Encapsulation)

ส่วนประกอบของวัตถุตามแนวความคิดของโปรแกรมเชิงวัตถุ จะต้องประกอบด้วย 2 ส่วน เป็นอย่างน้อย คือ ส่วนของคุณสมบัติใช้เก็บข้อมูลรายละเอียด สถานะ โดยจะใช้ตัวแปรเก็บค่าต่าง ๆ ไว้ และส่วนของเมธอดที่เป็นตัวเชื่อมการทำงานของวัตถุนั้น ๆ โดยผู้ใช้จะไม่สามารถติดต่อใช้งานตัวแปรที่อยู่ข้างในได้ ซึ่งในภาษาซีพลัสพลัส (C++) จะใช้คำว่า Public, Private และ Protected เข้ามาช่วยกำหนดขอบเขตการใช้งาน

การนำวัตถุมาใช้ใหม่ (Reuse the Object)

จุดประสงค์ใหญ่ของการเขียนโปรแกรมเชิงวัตถุ ก็คือ การนำส่วนต่างๆ ของวัตถุที่สร้างขึ้นมาใช้ใหม่หรือ ที่เรียกในภาษาอังกฤษว่า "Reuse" เมื่อผู้เขียนโปรแกรมสร้างวัตถุมีจำนวนมากพอก็สามารถนำวัตถุที่สร้างขึ้นมาประกอบเป็นวัตถุใหม่ หรือที่เรียกว่าคอมโพสิตชัน (Composition) ยกตัวอย่างเช่น การสร้างรถยนต์ ผู้ใช้ไม่จำเป็นจะต้องสร้างรถยนต์โดยเริ่มจาก ส่วนประกอบต่าง ๆ ที่ทำใหม่ทุกครั้ง แต่ผู้ใช้สามารถสร้างจาก ส่วนประกอบที่มีอยู่แล้วได้ นอกจากวิธีการคอมโพสิตแล้ว ผู้ใช้สามารถ Reuse ส่วนของวัตถุโดยใช้สืบทอดคุณสมบัติ (Inheritance) จากคลาส ลักษณะเช่นนี้ คือ เป็นการนำส่วนของวัตถุ ทั้งหมดมาใช้ โดยปกติแล้ววัตถุที่นำมาใช้ในลักษณะนี้จะมีขนาดใหญ่ ถ้าเป็นคอมโพสิตจะประกอบขึ้นจากส่วนของวัตถุที่มี ขนาดเล็กกว่า อย่างไรก็ตาม ขนาดของวัตถุไม่ได้เป็นตัวกำหนดที่แน่นอนตายตัวเสมอไป

การเขียนโปรแกรมและออกแบบระบบงาน

โดยปกติแล้วก่อนที่ผู้เขียนโปรแกรมจะสามารถเขียนคำสั่งได้ จะต้องมีการออกแบบระบบงานก่อนแล้วจึงเขียนโปรแกรมเป็น ภาษาต่าง ๆ ตามชนิดของงานและความเหมาะสม ในการเขียนโปรแกรมเชิงวัตถุก็เช่นเดียวกันจะต้องมีการออกแบบระบบงาน ก่อน หลักสำคัญสำหรับการออกแบบระบบงานแบบเชิงวัตถุ นั้น คือ การหาวัตถุให้พบ เมื่อพบวัตถุแล้วจะต้องจำแนกวัตถุ ออกเป็นส่วนที่เปลี่ยนแปลง และส่วนที่อยู่คงที่ วัตถุที่ไม่เปลี่ยนแปลงสามารถนำไปใช้ได้เมื่อมีการปรับปรุงระบบงานใหม่ ซึ่ง เป็นเหตุผลที่ทำให้ต้องมีการออกแบบระบบงาน วัตถุมีการเปลี่ยนแปลงบ่อย ซึ่งได้แก่ วัตถุที่ทำหน้าที่เป็นอินเทอร์เฟซ

เป็นต้นเปรียบเทียบแนวคิดระหว่างการเขียนโปรแกรมเชิงกระบวนการ และเชิงวัตถุ

ตัวอย่างตู้ขายเครื่องดื่มอัตโนมัติ

วิธีการคิดแบบการเขียนโปรแกรมเชิงกระบวนการ

เมื่อมีการหยอดเหรียญเข้าตู้

1. มีการแสดงผลชนิดของน้ำในตู้ที่สามารถเลือกซื้อได้
2. ตรวจสอบจำนวนน้ำกระป๋องที่มีอยู่ในตู้
3. รับผลการเลือกชนิดน้ำ
4. ส่งน้ำที่เลือกออกมาจากช่อง
5. จัดเก็บเงินเข้าระบบ
6. หากมีเงินทอน ให้ทอนเงินที่เหลือ ที่ช่องรับเงินทอน

วิธีการคิดแบบการเขียนโปรแกรมเชิงวัตถุ

ผู้ขายเครื่องดื่มนมอัตโนมัติ ประกอบด้วยส่วนประกอบต่าง ๆ ได้แก่ หน่วยตรวจสอบและจัดการเรื่องเงิน หน่วยจัดการเครื่องดื่ม หน่วยแสดงผล และรอรับคำสั่ง

1. หน่วยตรวจสอบและจัดการเรื่องเงิน มีข้อมูลเกี่ยวกับเงินที่ได้รับ และเงินที่มีอยู่ในระบบ สามารถ รับและตรวจสอบเงินที่หยอดเข้ามาได้ และทอนเงินได้
2. หน่วยจัดการเครื่องดื่ม มีข้อมูลชนิดของเครื่องดื่ม จำนวนเครื่องดื่ม สามารถจัดเตรียมชนิดเครื่องดื่มตามจำนวนเงินที่หยอด และสามารถจ่ายเครื่องดื่มออกมาจากตู้ได้
3. หน่วยแสดงผลและรอรับคำสั่ง มีหน้าที่รอรับคำสั่ง และแสดงผลเงินที่หยอดเข้ามาหมายเหตุ ตัวอย่างนี้เป็นเพียงตัวอย่างโดยสังเขป

แนวทางการออกแบบและแก้ปัญหา

การออกแบบและพัฒนาโปรแกรมเชิงวัตถุมีหลายด้าน โดยแนวทางดังต่อไปนี้ เป็นแนวทางที่ได้รับการยอมรับอย่างกว้างขวางในการใช้เพื่อแก้ไขปัญหา

ดีไซน์แพตเทิร์น-แบบแผนและแนวทางการออกแบบ

ในการออกแบบและการพัฒนาโปรแกรมเชิงวัตถุ ได้มีการรวบรวมบันทึกวิธีการแก้ปัญหาที่ใช้ได้ผลสำหรับปัญหาที่เกิดขึ้นซ้ำ ๆ เสมอ ๆ วิธีการแก้ไขเหล่านี้สามารถนำมาใช้ได้บ่อยๆ ในสถานการณ์ที่หลากหลาย บันทึกรวบรวมนี้มีชื่อเรียกเฉพาะว่า ดีไซน์แพตเทิร์น (Design Patterns) Design Patterns ซึ่งเป็นหนังสือที่ออกจัดจำหน่ายเมื่อปี 2538 โดยผู้แต่งร่วม 4 คน ได้แก่ Erich Gamma, Richard Helm, Ralph Johnson และ John Vissides หรือที่รู้จักในนามของ GoF (Gang of four) ถือว่าเป็นแบบแผนและแนวทางการออกแบบที่ได้รับความนิยมและเป็นที่รู้จักอย่างกว้างขวางในการนำมาประยุกต์ใช้งานจริง

การเขียนโปรแกรมเชิงวัตถุและฐานข้อมูล

การเขียนโปรแกรมเชิงวัตถุและระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management Systems) ได้ถูกใช้งานร่วมกันอย่างแพร่หลายในปัจจุบัน แต่เนื่องจากฐานข้อมูลเชิงสัมพันธ์ไม่สามารถเก็บข้อมูลเชิงวัตถุได้โดยตรง จึงมีความจำเป็นที่จะต้องเชื่อมต่อเทคโนโลยีทั้งสองเข้าด้วยกัน การแก้ปัญหาสองแบบที่ได้รับความนิยมแพร่หลายคือ การใช้ตัวส่งระหว่างโมเดลเชิงวัตถุและเชิงสัมพันธ์ (Object-Relational Mapping : ORM)

อีกวิธีการคือการใช้งานระบบจัดการฐานข้อมูลเชิงวัตถุ (Database Management Systems) แทนที่ ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ แต่วิธีการนี้ยังไม่ได้รับความนิยมมากนัก

โปรแกรมเชิงวัตถุและการเทียบเคียงกับโลกของความเป็นจริง

การเขียนโปรแกรมเชิงวัตถุสามารถนำมาใช้จำลองการทำงานตามโลกของความเป็นจริงได้ แต่ไม่ได้เป็น ที่นิยมนักเนื่องจากมี นักวิชาการจำนวนหนึ่งที่ไม่เห็นด้วยและมองว่าไม่ใช่วิธีการที่ถูกต้อง ปัจจุบันวิธีการมาตรฐานที่ใช้ในการเทียบเคียงกับโลกของ ความเป็นจริง ตามแนวทางของคณิตศาสตร์คือ Circle-ellipse problem ซึ่งก็ถูกต้องบางส่วน แต่แนวคิดการสร้างยังคงต้องให้ สอดคล้องกับความเป็นจริงของพื้นฐานธรรมชาติที่เป็นไปได้ผนวกกับคณิตศาสตร์ด้วย เพื่อให้เกิดสมดุล

1.4

คุณสมบัติของโปรแกรมเชิงวัตถุ

1. ความสามารถในการสืบทอด (Inheritance) คือ คุณสมบัติของการเขียน โปรแกรมเชิงวัตถุ (OOP) ที่เรา สามารถสร้างคลาสใหม่โดยสืบทอดจากคลาสเดิมได้ โดยคลาสที่ได้รับการสืบทอดจะได้รับตัวแปรและเมธอดจากคลาสหลักมา ใช้งาน และสามารถเพิ่มเติมการทำงานหรือความสามารถเข้าไปได้ นี่เป็นแนวคิดที่สำคัญของออกแบบ โปรแกรมให้นำโค้ด กลับมาใช้ใหม่ได้ (Reusable)

2. ความสามารถในการเก็บซ่อน (Encapsulation) คือ คุณสมบัติในการเขียน โปรแกรมเชิงวัตถุแล้วมีการ กำหนดการเข้าสมาชิกภายใน Class ไม่ว่าภายนอกหรือภายในก็ตามจะถูกนำไปใช้ เพื่อป้องกันข้อมูลภายในให้มีความปลอดภัย และเป็นความลับและง่ายต่อการเข้าใจในการเขียน โปรแกรม ยกตัวอย่าง Encapsulation จากชีวิตความเป็นจริง เช่น เมื่อเรา ต้องการจะกดน้ำเปล่าจากเครื่องกรอกน้ำเราต้องใส่เหรียญ 1 บาท เหรียญ 2 บาท เหรียญ 5 บาท และเหรียญ 10 บาท นั้น เพื่อที่จะ ได้น้ำตามปริมาณตามที่เราต้องการ และในการทำงานที่จะได้น้ำเปล่ามานั้นจะมีการทำงานที่อยู่ภายในเครื่องซ่อนอยู่โดยสิ่งที่ ปกปิดเราจะเรียกว่า Encapsulation คือ เราไม่จำเป็นต้องรู้วิธีการทำงานของเครื่องแต่สิ่งที่เราต้องรู้คือ การใช้งานของเครื่อง เพื่อที่จะได้น้ำเปล่าออกมา ภาษาของ Programming คือการใช้งาน Methods ของคลาสเพื่อวัตถุประสงค์บางอย่างและรู้ วิธีการ เรียกใช้งาน Encapsulation คือ คุณสมบัติในการเขียน โปรแกรมวัตถุโดยการกำหนดการเข้าถึงสมาชิกภายในและการป้องกัน ข้อมูลภายในให้มีความปลอดภัยและยังเก็บเป็นความลับ โดยการทำงานนั้นออกแบบมาให้ง่าย ต่อความเข้าใจในการเขียน โปรแกรม

3. ความสามารถในการพ้องรูป (Polymorphism) หรือในภาษาไทยเรียกว่า "การกำหนดรูปลหลายรูปแบบ" เป็นหนึ่งในแนวคิดหลักของการเขียน โปรแกรมแบบวัตถุ (Object-Oriented Programming-OOP) นอกเหนือจาก Encapsulation, Inheritance และ Abstraction. Polymorphism เป็นการสื่อสารที่ช่วยให้วัตถุคนละประเภทนั้น สามารถถูกใช้งาน ผ่าน interface เดียวกันได้ มันให้ความสามารถให้กับ โปรแกรมเมอร์ในการเขียน โค้ดที่มีความยืดหยุ่นและสามารถปรับใช้ได้กับ หลากหลายสถานการณ์ Polymorphism มาจากคำในภาษากรีกที่แปลว่า 'poly' หมายถึง "หลาย" และ 'morphism' หมายถึง "รูปร่าง" เมื่อนำมารวมกันคือการมีหลายรูปร่าง ในทางโปรแกรมมิ่ง คือ ความสามารถของฟังก์ชัน, วัตถุหรือตัวแปรในการรับ รูปร่างหรือแสดงพฤติกรรมต่าง ๆ ได้หลายแบบ

ประโยชน์ของ Polymorphism

ทำให้โค้ดมีความสะดวกในการจัดการ และสามารถนำกลับมาใช้ซ้ำได้ เปรียบเสมือนการทำงานที่มี"หน้าตา" ได้หลายแบบ แต่ใช้หลักการเดียวกันในการแก้ปัญหา คุณลักษณะนี้ไม่เพียงแต่ช่วยลดความซ้ำซ้อนของโค้ดเท่านั้น แต่ยังอำนวยความสะดวกในการหาสาเหตุของปัญหาและจัดการกับข้อผิดพลาดที่เกิดขึ้นอีกด้วย

4. ความสามารถในการจัดการโครงสร้างข้อมูลแบบเชิงนามธรรม (Abstract) แนวคิดเชิง

นามธรรม (Abstract Thinking) เป็นองค์ประกอบหนึ่งของแนวคิดเชิงคำนวณ (Computational Thinking) ซึ่งใช้กระบวนการ คัดแยกคุณลักษณะที่สำคัญออกจากรายละเอียดปลีกย่อยในปัญหา หรืองานที่กำลังพิจารณา เพื่อให้ได้ข้อมูล ที่จำเป็นและเพียงพอ ในการแก้ปัญหา แนวคิดเชิงคำนวณ (Computational Thinking) เป็นกระบวนการวิเคราะห์ปัญหา เพื่อให้ได้แนวทางหาคำตอบ อย่างเป็นขั้นตอนที่สามารถนำไปปฏิบัติได้โดยบุคคลหรือคอมพิวเตอร์อย่างถูกต้อง การคิดเชิงคำนวณเป็นกระบวนการ แก้ปัญหาในหลากหลายลักษณะ เช่น การจัดลำดับเชิงตรรกศาสตร์ การวิเคราะห์ข้อมูล และการสร้างสรรค์วิธีแก้ปัญหาไปทีละขั้น รวมทั้งการย่อยปัญหาที่ช่วยให้ง่ายขึ้นกับปัญหาที่ซับซ้อนหรือมีลักษณะเป็นคำถามปลายเปิดได้วิธีคิดเชิงคำนวณ จะช่วยทำให้ปัญหาที่ซับซ้อนเข้าใจได้ง่ายขึ้น เป็นทักษะที่เป็นประโยชน์อย่างยิ่งต่อทุก ๆ สาขาวิชา และทุกเรื่องในชีวิตประจำวันซึ่งไม่ได้จำกัดอยู่เพียงการคิดให้เหมือนคอมพิวเตอร์แต่เป็นกระบวนการคิดแก้ปัญหาของมนุษย์ เพื่อสั่งให้คอมพิวเตอร์ทำงานและช่วย แก้ปัญหาตามที่เราร้องการได้อย่างมีประสิทธิภาพ

แนวคิดเชิงคำนวณ

มีนักวิชาการได้กล่าวถึงนิยามของคำว่า แนวคิดเชิงคำนวณไว้มากมาย ดังนั้น ความหมายของคำว่าแนวคิดเชิงคำนวณ ได้ถูกถ่ายทอดออกมาหลายรูปแบบ แต่สิ่งที่เหมือนกัน คือ การนำแนวคิดเชิงคำนวณมาใช้ในการแก้ปัญหาเพื่อให้เกิดผลลัพธ์ของการแก้ปัญหาที่มีประสิทธิภาพ แนวคิดเชิงคำนวณ (Computational Thinking) คือ แนวคิดในการแก้ปัญหาลักษณะต่าง ๆ อย่างเป็นระบบเป็น กระบวนการที่มีลำดับขั้นตอนชัดเจน โดยกระบวนการแก้ปัญหาดังกล่าวนี้นี้เป็นกระบวนการที่ทั้งมนุษย์และคอมพิวเตอร์สามารถ เข้าใจร่วมกันได้ ซึ่งแนวคิดเชิงคำนวณเป็นแนวคิดสำคัญสำหรับการพัฒนาซอฟต์แวร์คอมพิวเตอร์ แต่สามารถนำมาประยุกต์ใช้ ในการแก้ปัญหาลักษณะต่าง ๆ ในชีวิตได้เช่นกัน แนวคิดเชิงคำนวณเป็นเครื่องมือในการแก้ปัญหาที่มีวิธีแก้ไขที่เป็นลำดับขั้นตอน มากกว่าเป็นการสร้างผลลัพธ์ แนวคิดลักษณะนี้ไม่เพียงนำไปใช้กับคอมพิวเตอร์ได้เท่านั้น แต่สามารถนำไปปรับใช้ได้กับทุก สถานการณ์เมื่อมีกระบวนการที่เป็นลำดับขั้นตอนเกิดขึ้นกับคอมพิวเตอร์ สิ่งที่เกิดขึ้นนี้เรียกว่า การเขียนโปรแกรมแต่ถ้า กระบวนการนั้นไม่ได้เกิดขึ้นจากแนวคิดเชิงคำนวณแล้ว ก็จะกลายเป็นโปรแกรมคอมพิวเตอร์ที่ทำงานซ้ำ

และทำให้ผู้ใช้งานผิดหวังเพราะทำงานไม่ตรงตามที่ต้องการ หลายกรณีระบบขึ้นช้ามาซึ่งใช้เวลานานในการตอบสนอง นั่นเป็น เพราะวิธีการออกแบบในบางจุดไม่มีประสิทธิภาพ หรือไม่ได้สร้างการเข้าถึงข้อมูลซึ่งรู้ว่าอยู่จุดใดให้มีประสิทธิภาพ

1. ความสามารถในการเรียกใช้ได้หลายครั้ง ออบเจกต์ได้ถูกออกแบบตามหลักการที่ว่าสามารถเรียกใช้งานได้หลาย ๆ ครั้ง ในหลักการนี้ทำให้แอปพลิเคชัน (Application) ของ OOP ตัวแรกอาจจะทำได้ยาก แต่ว่าโปรแกรมแอปพลิเคชันที่เขียนภายหลังจะสร้างง่ายเพราะสามารถเรียกใช้ออบเจกต์ที่ถูกสร้างไว้ตั้งแต่โครงงานแรกได้

2. ความเชื่อถือได้ โปรแกรมแอปพลิเคชันของ OOP จะมีความเชื่อถือได้สูง เพราะจะรวมเอาส่วนย่อย ที่ทดสอบจนได้มาตรฐานแล้วมารวมเข้าไว้ด้วยกัน รหัส (Code) ที่เขียนขึ้นมาใหม่ในแต่ละแอปพลิเคชันจะมีไม่มากนัก เนื่องจากรหัสส่วนใหญ่จะถูกดึงมาจากไลบรารีที่มีความเชื่อถือได้สูงอยู่แล้ว และในการเขียน โปรแกรมภาษาซีพลัสพลัส (C++) ยังมีคุณสมบัติประโยชน์อื่นอีก

3. ความต่อเนื่องกัน การพัฒนาซอฟต์แวร์แบบ OOP ในภาษาซีพลัสพลัส C++ จะเปลี่ยนไปตามฝีมือและจำนวนนักเขียนโปรแกรมภาษาซี (C) นักเขียนโปรแกรมภาษาซี (C) สามารถรู้หลักการของ OOP ได้ภายในเวลาไม่นาน และสามารถเข้าใจเนื้อหาได้ไม่ยาก อีกทั้งสามารถแปลงโปรแกรมแอปพลิเคชันของภาษาซี (C) เป็นภาษาซีพลัสพลัส (C++) ได้อย่างรวดเร็ว

สรุปสาระสำคัญ

การเขียนโปรแกรมเชิงวัตถุ หมายถึง วิธีการเขียนโปรแกรมของนักเขียนโปรแกรมรุ่นใหม่ ที่จัดแบ่งการเขียนคำสั่งกันออกเป็นชุด ๆ แต่ละชุดเรียกว่า "วัตถุ" (Object) แล้วจึงนำเอาชุดคำสั่งแต่ละชุดนั้นมารวมกันเป็นโปรแกรมชุดใหญ่อีกหนึ่งทีหนึ่ง ในบางครั้งยังอาจนำ "วัตถุ" ของโปรแกรมหนึ่งไปรวมกับ "วัตถุ" ของอีกโปรแกรมหนึ่ง แล้วเรียกออกมาใช้ได้เลย ทั้งนี้ ทำให้ผู้เขียนโปรแกรมใหม่ไม่จำเป็นต้องเริ่มต้นใหม่ทั้งหมด วิธีการดังกล่าว นี้ช่วยประหยัดเวลาได้มาก พูดให้ง่าย ก็คือ ทุกโปรแกรมไม่ต้องเริ่มต้นจากศูนย์ นั่นคือ คำว่า "วัตถุ" นั้น หมายถึงไปถึงภาพหรือกราฟิกด้วย ภาพหนึ่งภาพ เช่น การสร้างวงกลมนั้น เกิดจากการเขียนโปรแกรมโดยใช้สูตรคำนวณเส้นโค้ง ซึ่งจะประกอบด้วยคำสั่งหลายร้อยคำสั่ง แล้วเก็บไว้เป็น "วัตถุ" หนึ่ง ฉะนั้น เมื่อใดที่เราสั่งวาดวงกลม ก็เท่ากับไปเรียก "วัตถุ" นี้มาใช้ หลังจากนั้น, หากเราจะต่อเติมเป็นภาพอื่นต่อไป คอมพิวเตอร์ก็จะไปดึงอีก "วัตถุ" หนึ่งมาทำต่อให้

จะเห็นได้ว่าแนวคิดของ OOP นั้นจะต้องมีลักษณะที่คล้ายกับธรรมชาติของสิ่งหนึ่งโดยสามารถแยกสิ่งต่าง ๆ ออกเป็นแต่ละประเภทได้ ถ้านำเอาแนวคิดของ OOP มาใช้ในการเขียนโปรแกรมและในการจัดการ ข้อมูลนั้น จะเห็นได้ว่าโปรแกรมหรือฟังก์ชันแต่ละตัว ถึงแม้ว่าจะมาจากที่เดียวกันแต่สามารถทำงานในคนละหน้าที่ที่เก็บข้อมูลคนละค่าได้โดยจะไม่ยุ่งเกี่ยวกันแต่อย่างใด เนื่องจากหลักการเขียนโปรแกรมเชิงวัตถุเป็นแนวคิดแบบใหม่ ดังนั้น การทำงานหลาย ๆ ส่วนของการเขียนโปรแกรมแบบนี้ ซึ่งยังไม่เป็นที่คุ้นเคยมากนัก จึงควรมีความรู้เบื้องต้นเกี่ยวกับ OOP ดังนี้

การเชื่อมต่อ (Interface)

อินเทอร์เฟซ (Interface) หมายถึง การเชื่อมต่อ ถ้าการเชื่อมต่อระหว่างผู้ใช้กับคอมพิวเตอร์ จะเรียกการเชื่อมต่อนั้นว่า ยูสเซอร์ อินเทอร์เฟซ (User Interface) โดยปกติจะหมายถึง ส่วนของหน้าจอที่ผู้ใช้ส่งหรือ ติดต่อให้คอมพิวเตอร์ทำงาน แต่ในการเขียน โปรแกรมเชิงวัตถุ การเชื่อมต่อยังรวมไปถึงวัตถุ (Object) เพราะในวัตถุจะต้องมีอินเทอร์เฟซ ก็คือ การเปลี่ยนแปลงที่เกิดขึ้น ภายในวัตถุจะไม่กระทบต่ออินเทอร์เฟซ ดังนั้นภายในวัตถุผู้เขียนคำสั่งสามารถดัดแปลง แก้ไข หรือเพิ่มเติมได้ตลอดเวลา นอกจากนี้ ภายในวัตถุยังสามารถเก็บค่าต่าง ๆ โดยไม่จำเป็นต้องทราบกลไกการทำงานภายใน เมื่อใช้ก็เพียงแค่เรียกส่วนของ อินเทอร์เฟซ (หรือ ที่เรียกว่า Method) โดยการส่งข่าวสาร (Message) ไปยังวัตถุที่ต้องการ ตัวอย่างเช่น การทำงานของโทรศัพท์ กับรีโมท ซึ่งการใช้งานของโทรศัพท์มีดังต่อไปนี้ คือ เปิด ปิด เปลี่ยนช่องเมื่อผู้ใช้ต้องการเปิดสัญญาณ โดยกด ปุ่มสัญลักษณ์ เปิดส่งไปยังตัวรับโทรศัพท์ หลังจากนั้นโทรศัพท์ก็จะทำงานในกลไกของการเปิด ซึ่งเราไม่ต้องรับรู้ถึงวงจรที่อยู่ภายในเลย โดย เปิด ปิด และเปลี่ยนช่อง ก็คือ Method ของโทรศัพท์ เป็นต้น

การซ่อนรายละเอียด (Encapsulation)

ส่วนประกอบของวัตถุตามแนวความคิดของโปรแกรมเชิงวัตถุ จะต้องประกอบด้วย 2 ส่วน เป็นอย่างน้อย คือ ส่วนของ คุณสมบัติใช้เก็บข้อมูลรายละเอียด สถานะ โดยจะใช้ตัวแปรเก็บค่าต่าง ๆ ไว้ และส่วนของเมธอดที่เป็นตัวเชื่อมการทำงาน ของ วัตถุนั้น ๆ โดยผู้ใช้งานไม่สามารถติดต่อใช้งานตัวแปรที่อยู่ข้างในได้ ซึ่งในภาษา C++ จะใช้คำว่า Public, Private และ Protected เข้ามาช่วยกำหนดขอบเขตการใช้งาน

การนำวัตถุมาใช้ใหม่ (Reuse the Object)

จุดประสงค์ใหญ่ของการเขียน โปรแกรมเชิงวัตถุ ก็คือ การนำส่วนต่างๆ ของวัตถุที่สร้างขึ้นมาใช้ใหม่หรือ ที่เรียกใน ภาษาอังกฤษว่า "Reuse" เมื่อผู้เขียนโปรแกรมสร้างวัตถุมีจำนวนมากพอก็สามารถนำวัตถุที่สร้าง ขึ้นมาประกอบเป็นวัตถุใหม่ หรือที่เรียกว่าคอมโพสิชัน "composition" ยกตัวอย่างเช่น การสร้างรถยนต์ผู้ใช้ไม่จำเป็นจะต้องสร้างรถยนต์โดยเริ่มจาก ส่วนประกอบต่าง ๆ ที่ทำใหม่ทุกครั้ง แต่ผู้ใช้สามารถสร้างจาก ส่วนประกอบที่มีอยู่แล้วได้ออกจากวิธีการคอมโพสิตแล้ว ผู้ใช้ สามารถ Reuse ส่วนของวัตถุโดยใช้สืบทอดคุณสมบัติ (Inheritance) จากคลาส ลักษณะเช่นนี้ คือ เป็นการนำส่วนของวัตถุ ทั้งหมดมาใช้ โดยปกติแล้ววัตถุที่นำมาใช้ในลักษณะนี้จะมีขนาดใหญ่ ถ้าเป็นคอมโพสิตจะประกอบขึ้นจากส่วนของวัตถุที่มี ขนาดเล็กกว่า อย่างไรก็ตามขนาดของวัตถุไม่ได้เป็นตัวกำหนดที่แน่นอนตายตัวเสมอไป